Python Exceptions and Debugging

Python provides a way to handle these errors (Exceptions) and tools to find their causes (Debugging).

Reference: https://realpython.com/python-exceptions/

The Exception Handling Block

Python uses a `try–except` block to manage errors without crashing the program.

- `try` : The code that might cause an error.
- `except` : The code that runs if an error occurs.
- `else` : Runs ONLY if no errors occurred in the try block.
- `finally` : Runs no matter what (useful for closing files or cleanup).

In [...

```
num = 'a'
100/num
```

```
---------------------------------------------------------
---------------------
TypeError                                 Traceback
(most recent call last)
Cell In[9], line 2
      1 num = 'a'
----> 2 100/num

TypeError: unsupported operand type(s) for /: 'int' a
nd 'str'
```

In [...

```
a = 10
try:
    num = 'a'
    # num = float(input("Enter a number to divide 100
    # type(num)
    result = 100 / num
except TypeError:
    print("unsupported operand type(s) for /: 'int' a
```

```
except ValueError:
    print("Error: That wasn't a valid number!")
except ZeroDivisionError:
    print("Error: You cannot divide by zero!")
else:
    print(f"Success! The result is {result}")
finally:
    print("Execution complete. Cleaning up resources.
```

```
unsupported operand type(s) for /: 'int' and 'str'
Execution complete. Cleaning up resources...
```

In […

```
int(20.0)
```

Out[…

```
20
```

In […

```
# num = float(input())
num = float(input())
if type(num) == str:
    print('you entered a string!')
else:
    print(100/num)
```

```
10.0
```

In […

```
# a = 2
# 2 + a
# '2' + 'a'

lst = [1,2,3]
for i in range(5):
    print(lst[i]) #index error
```

```
1
2
3
```

```
---------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
Cell In[15], line 7
      3 # '2' + 'a'
      4
      5 lst = [1,2,3]
      6 for i in range(5):
----> 7     print(lst[i]) #index error

IndexError: list index out of range
```

Common Python Exceptions

Recognizing these common errors helps you debug faster:

Exception	Cause
`TypeError`	Operation applied to an object of inappropriate type (e.g., `'a' + 5`).
`ValueError`	Argument has right type but inappropriate value (e.g., `int('hello')`).
`IndexError`	Trying to access a list index that doesn't exist.
`KeyError`	Trying to access a dictionary key that doesn't exist.

In [...
```
d = {'a': 1, 'b':2}
d['a']
d['c']
```

Out[...
```
1

---------------------------------------------------------------------
KeyError                                  Traceback
(most recent call last)
Cell In[16], line 3
      1 d = {'a': 1, 'b':2}
      2 d['a']
----> 3 d['c']

KeyError: 'c'
```

Tracebacks

A Traceback is the report Python gives you when code fails. Read it from bottom to top:

1. The last line tells you the error type and a brief description.
2. The lines above show exactly where the error happened (File name and Line number).

Debugging Basics

The "Print" Method

The simplest way to debug is to print variable states at different stages to see where they deviate from your expectations.

Inspection Tools

- `type(obj)` : Check what kind of data you are working with.
- `len(obj)` : Check the size of a collection to avoid `IndexError` .

In [...
```
def calculate_average(data):
    print(f"DEBUG: data type is {type(data)}, length
    total = sum(data)
```

```
    return total / len(data)


my_data = [10, 20, 30]
my_data
print(calculate_average(my_data))
```

```
Out[…  [10, 20, 30]
DEBUG: data type is <class 'list'>, length is 3
20.0
```

Advanced tools: pdb and IDEs (breakpoints).

Typically physicists don't use them; we are happy with our own `print` debugging. :P But look them up.